Projet d'IA : AlphaZero appliqué au Puissance 4

Houssam-Eddine FIDANI

Table des matières

1 Introduction **2**

2 Objectif du projet **2**

3 Structure du code **2**

4 Lancer le projet **2**

5 Étapes d'implémentation **3**
5.1 Implémentation du jeu 3
5.2 Réseau politique-valeur 3
5.3 Recherche MCTS 3
5.4 Auto-jeu et génération des exemples 3
5.5 Apprentissage 4

6 Résultats **4**
6.1 Comparaison des configurations 4
6.2 Tests des modèles 5

7 Interface graphique **5**

8 Limites **5**

9 Conclusion **5**

1 Introduction

AlphaZero est un algorithme d'apprentissage par renforcement pour les jeux à deux joueurs. Un agent joue contre lui-même, produit des exemples, puis améliore un réseau de neurones avec ces exemples.

Le réseau prend un état du jeu en entrée. Il retourne deux informations :

$$(P, V) = f_\theta(s)$$

— **P** : une politique, c'est-à-dire une préférence pour chaque action possible
— **V** : une estimation de la valeur de la position.

AlphaZero ne joue pas directement le coup proposé par le réseau. Il utilise une recherche **Monte Carlo Tree Search** (MCTS), guidée par ce réseau. MCTS retourne une politique π, calculée à partir du nombre de visites des coups.

Le jeu choisi est le Puissance 4. Les règles sont les suivantes :
— le plateau contient 6 lignes et 7 colonnes
— deux joueurs jouent chacun leur tour
— un pion tombe dans la case libre la plus basse de la colonne choisie
— un joueur gagne s'il aligne 4 pions horizontalement, verticalement ou en diagonale
— si le plateau est plein sans gagnant, la partie est nulle.

2 Objectif du projet

L'objectif est de réaliser un **prototype AlphaZero** pour le Puissance 4. Les idées principales sont :
— représentation d'un jeu à deux joueurs
— auto-jeu
— recherche MCTS
— réseau de neurones avec deux sorties
— apprentissage à partir des exemples produits par MCTS.

3 Structure du code

`Fichier`	Rôle
`puissance4.py`	Règles du jeu.
`model.py`	Réseau politique-valeur.
`mcts.py`	Recherche MCTS.
`apprentissage.py`	Auto-jeu et entraînement.
`test.py`	Évaluation des modèles.
`experiences.py`	Comparaison des configurations.
`interface.py`	Interface graphique.

Tableau 1 : rôle des fichiers du projet.

4 Lancer le projet

Depuis le dossier `src`, les commandes principales sont :

`Commande`	Rôle	Sortie
`python experiences.py`	Reproduit les entraînements et les tests.	`model_1.pth`, `model_2.pth`, `model_3.pth`, `loss_1.png`, `loss_2.png`, `loss_3.png`, `resultats_experiences.txt`.
`python interface.py`	Lance le jeu contre l'IA.	Fenêtre graphique utilisant `model_3.pth`.

Tableau 2 : commandes principales et sorties.

5 Étapes d'implémentation

5.1 Implémentation du jeu

Le plateau est un tableau `numpy` de taille 6×7.
— `1` représente le joueur 1
— `-1` représente le joueur 2
— `0` représente une case vide.
Les fonctions principales sont :
— `legal_actions` : retourne les colonnes non pleines
— `play` : joue un coup en respectant la gravité
— `winner` : détecte les alignements de 4
— `resultat` : retourne 1 (victoire), 0 (nulle) ou -1 (défaite) du point de vue d'un joueur
— `encode_state` : prépare l'état pour le réseau.

5.2 Réseau politique-valeur

L'entrée du réseau contient deux plans 6×7 :
— un plan pour les pions du joueur courant
— un plan pour les pions de l'adversaire.
La taille d'entrée est :

$$2 \times 6 \times 7 = 84$$

Le réseau contient deux couches cachées de taille 64. Il produit :
— une sortie politique de taille 7
— une sortie valeur de taille 1, ramenée entre -1 et 1 par une tangente hyperbolique.
Dans le code, les deux sorties sont produites ainsi :

```
p = self.policy(x)
v = torch.tanh(self.value(x))
return p, v
```

5.3 Recherche MCTS

MCTS utilise quatre quantités :
— $N(s, a)$: nombre de visites du coup a depuis l'état s
— $W(s, a)$: somme des valeurs observées
— $Q(s, a)$: valeur moyenne du coup
— $P(s, a)$: probabilité initiale donnée par le réseau.
À chaque simulation, le coup maximise un score de type :

$$Q(s, a) + U(s, a)$$

La politique π vient des visites à la racine :

$$\pi_a = \frac{N(s, a)}{\sum_b N(s, b)}$$

Cette politique sert de cible d'apprentissage.

5.4 Auto-jeu et génération des exemples

Dans `apprentissage.py`, le modèle joue contre lui-même. Les deux joueurs utilisent le même réseau et MCTS.
Pour chaque position rencontrée, on stocke :

$$(s, \pi, z)$$

— s : l'état du plateau
— π : la politique produite par MCTS
— z : le résultat final de la partie, vu depuis le joueur qui devait jouer dans l'état s.

Les expériences utilisent **trois configurations**. Elles augmentent le nombre de parties d'auto-jeu, le nombre de simulations MCTS et le nombre d'époques.

Configuration	Parties	Simulations MCTS	Époques
config 1	100	50	10
config 2	200	100	20
config 3	**300**	**150**	**50**

Tableau 3 : paramètres des trois entraînements.

La taille de batch vaut 64. Le taux d'apprentissage vaut 0.001.

5.5 Apprentissage

La fonction de perte suit l'idée de l'article sur AlphaZero :

$$l = (z - v)^2 - \pi^T \log(p)$$

Dans le code, elle est séparée en deux termes :
— `value_loss` : erreur entre le résultat final z et la valeur prédite v
— `policy_loss` : erreur entre la politique MCTS π et la politique prédite p.

```
policy_loss = -mean(sum(pi * log_p))
value_loss = mean((z - v)^2)
loss = value_loss + policy_loss
```

Les modèles sont sauvegardés dans :

`model_1.pth`, `model_2.pth`, `model_3.pth`

6 Résultats

6.1 Comparaison des configurations

Les trois configurations produisent une baisse de la **perte moyenne**. La **configuration 3** donne la perte finale la plus basse.

Configuration	Parties	Simulations	Époques	Perte finale
config 1	100	50	10	2.653829
config 2	200	100	20	2.633240
config 3	**300**	**150**	**50**	**2.347066**

Tableau 4 : perte finale pour chaque configuration.

Les courbes montrent une diminution régulière. Cela indique que le réseau apprend à imiter MCTS et à prédire les résultats.

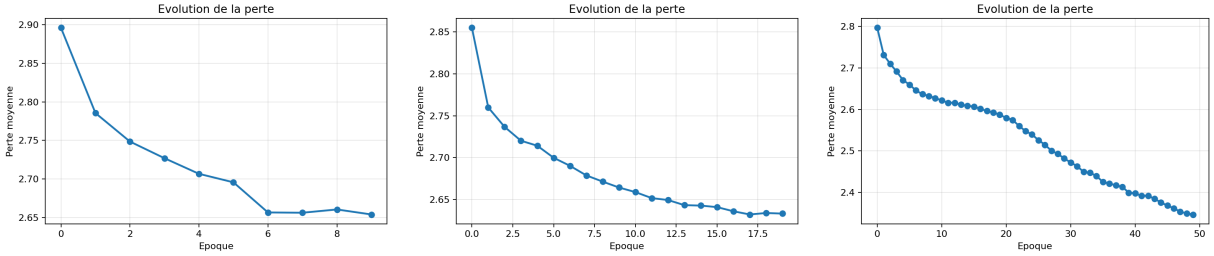


Figure 1 : évolution de la perte pour les configurations 1, 2 et 3.

6.2 Tests des modèles

Chaque modèle est ensuite testé avec 50 parties. Deux tests sont utilisés :
— **test aléatoire** : l'adversaire joue un coup légal au hasard
— **test amélioré** : l'adversaire joue un coup gagnant s'il existe, sinon il bloque une victoire immédiate, sinon il joue au hasard.
Dans le tableau, J1 signifie que l'IA commence la partie. J2 signifie que l'IA joue en deuxième.

Configuration	Test	Victoire J1	Nul J1	Victoire J2	Nul J2
config 1	aléatoire	1.00	0.00	1.00	0.00
config 1	amélioré	0.58	0.12	0.44	0.10
config 2	aléatoire	0.96	0.02	1.00	0.00
config 2	amélioré	0.66	0.16	0.68	0.10
config 3	aléatoire	**1.00**	0.00	0.96	0.00
config 3	amélioré	**0.76**	0.06	0.64	0.08

Tableau 5 : taux de victoire et de nul contre les deux tests.

Les trois modèles gagnent largement contre le **test aléatoire**. Le **test amélioré** est plus difficile. La **configuration 3** donne le meilleur résultat en joueur 1 et reste forte en joueur 2.

7 Interface graphique

L'interface `pygame` permet à un utilisateur de jouer contre l'IA. Elle charge `model_3.pth` et utilise MCTS pour choisir les coups de l'IA.
La touche `R` relance une partie.

8 Limites

Principales limites :
— le nombre de parties d'entraînement est faible par rapport à AlphaZero
— le réseau est très petit
— même la meilleure configuration reste limitée à 150 simulations MCTS par coup pendant l'entraînement
— il n'y a pas de cycles répétés auto-jeu puis réentraînement
— l'évaluation ne couvre pas un joueur optimal
— contre un joueur humain de niveau normal, le **modèle 3**, le plus performant, gagne **2 parties sur 5**.

9 Conclusion

Le projet implémente une version simplifiée d'AlphaZero pour le Puissance 4 : règles du jeu, réseau politique-valeur, MCTS, auto-jeu, apprentissage, évaluation et interface graphique.
Le modèle apprend à partir de l'auto-jeu. La **configuration 3** obtient la perte finale la plus basse et les meilleurs résultats globaux.
Les pistes d'amélioration sont claires : plus de cycles d'entraînement, plus de simulations MCTS, et une évaluation contre des joueurs plus forts.